

RAG and RIS Implementation Bundle

Overview: Retrieval-Augmented Generation (RAG) and Remote Instruction Stub (RIS) are complementary architectures designed to separate an LLM's knowledge retrieval from its logical instruction execution. **RAG** is a hybrid approach that integrates a retrieval mechanism with a generative model – a retrieval component fetches relevant information from external knowledge bases and supplies this context to the LLM for response generation ¹. This helps ground the model's output in up-to-date, factual data and reduce hallucinations. **RIS**, in contrast, handles the logical sequence of instructions (the “script” or process the LLM should follow) and is kept separate from raw data. This separation ensures the model “exclusively executes its primary instruction” while treating retrieved documents purely as passive data ², mitigating the risk of the model misinterpreting data as new commands (a common cause of prompt injection issues).

Below, we present a comprehensive, vendor-neutral implementation bundle for RAG and RIS. This includes repository layouts, schema definitions, manifest examples, templates, and guides for quality assurance, security, maintenance, and adoption. The bundle is designed for open systems with no proprietary dependencies, so it can be adopted in any environment. All example files are provided in a copy-paste-ready format, and the content is organized with academic clarity.

RAG Repository Structure

The RAG repository contains all assets related to **retrieval-augmented content** (knowledge bases, documents, indexes, etc.) that the LLM can draw upon. A typical layout might be:

- `rag/` – Root directory for the RAG bundle.
- `README.md` – Documentation on RAG usage and how to integrate the retrieval components into the LLM runtime.
- `index.json` – Manifest listing all content files and metadata (following the `rag_manifest.schema.json` format).
- `data/` – Directory for knowledge content files (e.g. Markdown or text documents). *Example files:* `company_policies.md`, `reference_articles.md`, etc.
- `embeddings/` – (Optional) Precomputed vector index or embedding files for the content (e.g. `index.faiss` or chunk embeddings in JSON). This allows efficient similarity search over `data/`.
- `schemas/` – (Optional) Schema definitions for validation (e.g. could include `rag_manifest.schema.json` for the manifest format).
- `QA/` – (Optional) Quality assurance scripts or data specific to RAG content (e.g. factual consistency checks).
- (Additional subfolders as needed for domain-specific data, images, etc. can be included, referenced in the manifest.)

Description: The RAG repository is primarily about **content**. The `data/` files are the knowledge sources the LLM will retrieve from. These could be text chunks, documents, or other media (though non-text might require special handling). The `index.json` manifest provides a roadmap of this content to the system: listing each document, possibly its topics or embedding references, and checksums for integrity. By keeping

all retrieval content under `rag/`, we ensure that knowledge can be updated or replaced independently of the LLM's logical instructions. This repository does **not** contain actual model code, only data and indexes. The LLM or an orchestrator will use this data (e.g., perform similarity search on the `embeddings/` index or scan `data/` files) to fetch relevant context for a query ¹.

RIS Repository Structure

The RIS repository contains all assets related to **instruction logic and process flows** that guide the LLM's behavior. Its layout might be:

- `ris/` – Root directory for the RIS bundle.
- `README.md` – Documentation on RIS usage and how to integrate these instruction stubs into the LLM runtime.
- `index.json` – Manifest listing all instruction files and templates (following the `ris_manifest.schema.json` format).
- `patterns/` – Structured process patterns in JSON form. These define multi-step instruction flows or decision logic. *Example:* `example_process.json` (see below) which outlines an interview process flow.
- `prompts/` – Prompt scaffolds in Markdown that correspond to patterns or specific guided tasks. *Examples:* `interview_flow.md`, `revision_flow.md`, `narrative_frame.md` (see examples below).
- `templates/` – Visual or structured templates that the LLM can use for formatting outputs. *Examples:* `swimlane_skeleton.mmd` (Mermaid diagram template), `minimal.bpmn` (BPMN process XML template), `basic_sequence.puml` (PlantUML sequence diagram template).
- `checklists/` – Checklists or quality criteria (in YAML/JSON) for outputs. *Example:* `diagram_quality.yaml` defines rules to ensure generated diagrams meet certain standards.
- `behavior/` – Configuration of output behavior and formatting rules. *Example:* `outputs.yaml` can specify preferences like answer format, tone, or post-processing policies.
- `schemas/` – (*Optional*) Schema definitions for RIS structures (e.g. `ris_manifest.schema.json`, `process_pattern.schema.json`, `qa_rules.schema.json` for validation and standardization).
- `QA/` – (*Optional*) Scripts or additional rules for validating that the LLM followed the instructions correctly (overlaps with `checklists/` and `QA.md` documentation).

Description: The RIS repository contains **logic and instruction assets**. The content here is not user-facing knowledge, but rather meta-content that shapes *how* the LLM uses knowledge or interacts. By externalizing these instruction patterns and templates, the system can enforce a clear separation between *commands* and *data*, improving safety ². For example, a pattern in `patterns/example_process.json` might define a step-by-step approach for an interview simulation, and `prompts/interview_flow.md` provides the natural language prompt template for that process. The LLM runtime will use the RIS bundle to determine how to structure its reasoning or output (e.g., which pattern to follow, which template to apply), independent of any particular knowledge content. This makes the logic reusable and easier to maintain or audit. It also allows changing how the LLM behaves (by updating instructions in RIS) without touching the RAG data, and vice versa.

RAG README (Usage in LLM Runtime)

Below is an example `README.md` for the RAG repository, describing its purpose and how to use the RAG bundle in an LLM pipeline:

Retrieval-Augmented Generation (RAG) Bundle

This repository provides a knowledge base and index for retrieval-augmented generation. The goal is to supply **external context** to a Large Language Model (LLM) at query time, improving factual accuracy and reducing hallucinations ¹.

Contents

- **``data/``**: Source knowledge documents (text, markdown, etc.) that the LLM can reference.
- **``index.json``**: A manifest listing all documents and their metadata (see **Manifest Format** below).
- **``embeddings/``**: (Optional) Precomputed embedding index for the documents, enabling fast similarity search.

Usage

1. **Indexing**: Before using the RAG bundle, index the documents in ``data/`` using an embedding model of your choice. You can use an open-source vector index (e.g., FAISS or similar) to create ``embeddings/index.bin``. Ensure the indexing process records document identifiers that match those in ``index.json``.
2. **Retrieval**: At runtime, when the user asks a question, your system should:
 - Convert the user query into an embedding (using the same embedding model).
 - Search the vector index in ``embeddings/`` for relevant documents.
 - Retrieve the top relevant passages or files from ``data/`` (the ``index.json`` can map IDs to file paths and even byte offsets if using segmented documents).
3. **Augmenting the Prompt**: Combine the retrieved text with the original query. Typically, the retrieved content (or a summarized form) is appended to the LLM prompt as contextual information. For example:

```
``plaintext
User question: "<user query here>"
Relevant context:
"<retrieved text snippet 1>"
"<retrieved text snippet 2>"
...
`
```

By grounding the LLM's input in this context, the model can generate a more accurate answer ¹.

4. **LLM Response**: The LLM processes the query along with the provided context and produces a response. Ideally, the model will base its answer on the given documents (reducing the chance of unsupported claims).

5. **Post-Processing**: Optionally, you can include source citations in the answer pointing to the documents (using the metadata from `index.json`). This can be done by instructing the LLM to output reference tags (for instance, document IDs or titles from the manifest).

Manifest Format

The `index.json` manifest describes the content in this bundle. It includes fields like:

- **bundle_name**, **version**: Identifier and version of this RAG bundle.
- **documents**: An array of document entries with at least:
 - `id`: A unique identifier for the document.
 - `file`: Path to the document file in `data/`.
 - `title` (optional): Human-readable title of the document.
 - `topics` (optional): Keywords or categories for the document.
 - `checksum` (optional but recommended): Hash of the file content for integrity verification.

A full JSON schema is provided in `schemas/rag_manifest.schema.json`. The manifest ensures that the retrieval system can programmatically load and verify all documents.

Considerations

- **Vendor Neutral**: This bundle does not depend on any proprietary service. You can use any open-source embedding model and vector store to index and retrieve documents.
- **Updating Data**: When updating or adding documents in `data/`, update `index.json` accordingly and increment the bundle version. Re-run the embedding indexing on the new data.
- **Data Limits**: Keep individual files reasonably sized (e.g., splitting long documents into sections) to optimize retrieval granularity. This helps ensure that any given retrieved chunk can fit in the LLM's prompt context.
- **Security**: Treat the retrieved data as read-only context. The LLM should not execute instructions found in `data/` ². (For example, if a document contains text like "delete record", the LLM must not act on it as a command—RIS logic should guard against this.)

(The above is a simplified README for RAG. In practice, it would also include examples of queries and answers, pointers on how to fine-tune the retrieval (e.g., number of documents to fetch), and any domain-specific details.)

RIS README (Usage in LLM Runtime)

Below is an example `README.md` for the RIS repository, describing how the instruction stubs are used with an LLM:

Remote Instruction Stub (RIS) Bundle

This repository provides structured **instructional logic** for the LLM, separate from the knowledge content. By isolating process flows and templates here, we ensure the model follows specific reasoning patterns while keeping data and instructions separate ².

Contents

- **patterns/**: JSON files defining step-by-step processes or decision trees (logic templates for the LLM to follow).
- **prompts/**: Markdown prompt scaffolds that correspond to the patterns, written in natural language. These serve as injectable templates or guidelines for the model.
- **templates/**: Diagram or output templates (Mermaid, BPMN, PlantUML, etc.) that the model can fill in when a visual or structured output is needed.
- **checklists/**: Quality checklists (in YAML) that enumerate criteria for specific outputs (e.g., what constitutes a "good" diagram).
- **behavior/**: YAML files defining output behavior rules or preferences (tone, format, length limits, etc.).
- **index.json**: Manifest listing all above files with metadata (see **Manifest Format** below).

Usage

1. **Selecting a Pattern**: When a user request comes in, determine if a specialized instruction pattern should be applied. For example, if the user asks "Can you interview me on topic X?", the system should load the **Interview** pattern (`patterns/interview_process.json`) and corresponding prompt scaffold (`prompts/interview_flow.md`). If no specific pattern fits, a default Q&A logic (perhaps a simple one-turn reply pattern) is used.
2. **Applying the Prompt Scaffold**: Insert any user-specific details into the selected prompt template. For instance, in an interview scenario, the `interview_flow.md` scaffold expects a **Topic** placeholder to be filled. The system (or the LLM itself in a preliminary step) replaces placeholders like `[[Topic]]` with the actual topic requested by the user.
3. **Executing the Pattern**: Guide the LLM through the steps defined in the JSON pattern:
 - Some patterns may involve multiple turns. For example, an interview pattern

might instruct the LLM to ask a question, wait for an answer (from the user or from another component), then ask a follow-up, etc.

- The orchestration logic can be implemented in the application: e.g., a loop that feeds the LLM each prompt step and collects responses. Alternatively, the LLM might be prompted to **internalize** the pattern (via the scaffold which lists the steps as role instructions) and carry out the entire sequence in one go.

- The **Remote Instruction Stub** acts like a remote "script". The LLM should **not deviate** from this script - it treats it as authoritative instructions.

4. **Integration with RAG**: Patterns can indicate points where retrieval is needed. For example, a pattern for a **research assistant** might have a step: "Search the knowledge base for information on `[[Query]]`". The application can catch such a step and invoke the RAG system to fetch data, then supply that data in the next LLM prompt. In this way, RIS and RAG work in concert (RIS decides **when** and **how** to retrieve; RAG provides the actual content).

5. **Ensuring Compliance**: After the LLM produces an output following the pattern, use the checklists in `checklists/` and any rules in `behavior/outputs.yaml` to validate the result. For example, if the pattern was supposed to produce a Mermaid diagram, apply `diagram_quality.yaml` rules to ensure the diagram is well-formed. This can be done by an automated validator or by an LLM function that inspects the output.

Manifest Format

The `index.json` file enumerates all components of the RIS bundle:

- **bundle_name**, **version**: Identifier and version of this RIS bundle.
- **patterns**: List of pattern JSON files and brief descriptions.
- **prompts**: List of prompt templates.
- **templates**: List of visual/text templates available.
- **checklists**: List of checklist/QA files.
- **behavior**: List of behavior/config files (often just one, like `outputs.yaml`).

A detailed schema is available in `schemas/ris_manifest.schema.json`. This manifest allows the application or LLM to discover what tools/instructions are available. For example, an automated system could read `index.json` to know that there's an interview flow pattern and when to use it (perhaps indicated by a description or tags).

Guidelines for Developing New Patterns

- **Modularity**: Each pattern (in `patterns/`) should represent a reusable logic segment (e.g., an interview, a text revision workflow, a storytelling framework). Avoid mixing unrelated tasks in one pattern.
- **Correspondence**: Usually, each pattern has a matching prompt scaffold in `prompts/`. The JSON defines the logic, while the Markdown provides the

phrasing. Keep them in sync (e.g., if the pattern has 5 steps, the prompt scaffold should prepare the model for those 5 steps).

- ****No Proprietary Calls****: The patterns should rely only on the open data in RAG and general LLM capabilities. If a step requires an external tool or API, design it such that it can be fulfilled with open components (or clearly document the requirement).
- ****Testing****: After creating a pattern and prompt, test it with various inputs. Ensure the LLM's behavior aligns with expectations (see ``Adoption_Playbook.md`` for test case ideas).

Safety and Integrity

By using RIS, we enforce a clear boundary between instructions and data. The LLM is less likely to confuse user-provided content with system instructions ². All the logic here can be audited independently: one can review a JSON pattern or a prompt template to see what the LLM is being asked to do. This transparency is an advantage of the RIS approach, aiding debugging and safety reviews.

(The above README for RIS covers how to use instruction bundles. In a real setup, it might also include a table of available patterns and their purposes, examples of input/output using each pattern, and advanced notes on customizing or extending the logic.)

Schema Definitions

To ensure consistency and interoperability, we define JSON schemas for manifests and key files. These schemas can be used to validate the bundle's JSON/YAML files. All schemas are vendor-neutral and use standard JSON Schema syntax (draft 7 or later).

1. `rag_manifest.schema.json` – Schema for the RAG bundle manifest (`rag/index.json`):

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "RAG Manifest Schema",
  "type": "object",
  "required": ["bundle_name", "version", "documents"],
  "properties": {
    "bundle_name": {
      "type": "string",
      "description": "Name of the RAG bundle (e.g., 'ACME_KnowledgeBase')"
    },
    "version": {
      "type": "string",
      "description": "Version of the bundle, following semver (X.Y.Z) 3 "
    },
    "schema_version": {
      "type": "string",

```

```

    "description": "Version of this manifest schema (for backward
compatibility)"
  },
  "description": {
    "type": "string",
    "description": "Brief description of the bundle contents or purpose"
  },
  "documents": {
    "type": "array",
    "description": "List of documents available for retrieval",
    "items": {
      "type": "object",
      "required": ["id", "file"],
      "properties": {
        "id": {
          "type": "string",
          "description": "Unique document identifier"
        },
        "file": {
          "type": "string",
          "description": "Path to the document file within the bundle"
        },
        "title": {
          "type": "string",
          "description": "Human-readable title of the document"
        },
        "topics": {
          "type": "array",
          "items": { "type": "string" },
          "description": "Topics or tags for the document (for filtering/
classification)"
        },
        "checksum": {
          "type": "string",
          "description": "Hash (e.g., SHA-256) of the document file for
integrity verification"
        }
      }
    }
  }
}

```

This schema ensures that the RAG manifest contains all required fields and that each document entry has at least an `id` and a `file` path. Additional metadata like `title`, `topics`, and `checksum` help with organization and security (the `checksum` can be used to verify the file's integrity as described in [Security_Integrity.md](#)). The version field should follow semantic versioning rules ³.

2. `ris_manifest.schema.json` – Schema for the RIS bundle manifest (`ris/index.json`):

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "RIS Manifest Schema",
  "type": "object",
  "required": ["bundle_name", "version"],
  "properties": {
    "bundle_name": {
      "type": "string",
      "description": "Name of the RIS bundle (e.g., 'ACME_LogicPatterns')"
    },
    "version": {
      "type": "string",
      "description": "Version of the bundle (semver format) 3 "
    },
    "schema_version": {
      "type": "string",
      "description": "Manifest schema version"
    },
    "patterns": {
      "type": "array",
      "description": "List of logic pattern files with metadata",
      "items": {
        "type": "object",
        "required": ["file"],
        "properties": {
          "file": { "type": "string", "description": "Path to a JSON pattern file" },
          "id": { "type": "string", "description": "Pattern identifier (unique key)" },
          "purpose": { "type": "string", "description": "Brief description of when/why to use this pattern" }
        }
      }
    },
    "prompts": {
      "type": "array",
      "description": "List of prompt template files",
      "items": {
        "type": "object",
        "properties": {
          "file": { "type": "string", "description": "Path to a prompt Markdown file" },
          "pattern_id": { "type": "string", "description": "ID of the pattern this prompt corresponds to (if any)" },
          "description": { "type": "string", "description": "Brief description"

```

```

of the prompt's role" }
    }
  },
  "templates": {
    "type": "array",
    "description": "List of reusable output templates (diagrams, etc.)",
    "items": {
      "type": "object",
      "properties": {
        "file": { "type": "string", "description": "Path to a template
file" },
        "type": { "type": "string", "description": "Template format (e.g.,
'mermaid', 'bpmn', 'puml')" },
        "description": { "type": "string", "description": "What this template
represents" }
      }
    },
  },
  "checklists": {
    "type": "array",
    "description": "List of QA checklist files",
    "items": {
      "type": "object",
      "properties": {
        "file": { "type": "string", "description": "Path to a checklist (YAML/
JSON) file" },
        "scope": { "type": "string", "description": "What output or aspect
this checklist applies to (e.g., 'diagram')" }
      }
    },
  },
  "behavior": {
    "type": "array",
    "description": "List of behavior/configuration files",
    "items": {
      "type": "object",
      "properties": {
        "file": { "type": "string", "description": "Path to a behavior config
file" },
        "scope": { "type": "string", "description":
"Type of behavior settings (e.g., 'output_format')" }
      }
    },
  },
}

```

This schema organizes the different categories of files in the RIS bundle. It's flexible: if a category (like `templates` or `checklists`) is not used, it can be omitted or left empty. Each entry can carry a description to aid human developers (and potentially an AI system in understanding when to apply each component). The manifest allows automated loading of all relevant instructions when initializing the system. For example, the application could read `ris/index.json` on startup to load all prompt templates into memory, or to register all available patterns.

3. `process_pattern.schema.json` - Schema for a process pattern JSON (e.g., `patterns/example_process.json`):

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "Instruction Process Pattern Schema",
  "type": "object",
  "required": ["pattern_id", "steps"],
  "properties": {
    "pattern_id": {
      "type": "string",
      "description": "Unique identifier for this pattern"
    },
    "purpose": {
      "type": "string",
      "description": "Short description of the pattern's purpose or scenario"
    },
    "steps": {
      "type": "array",
      "description": "Ordered list of steps in this instruction process",
      "items": {
        "type": "object",
        "required": ["id", "instruction"],
        "properties": {
          "id": {
            "type": "string",
            "description": "Step identifier or label (unique within the pattern)"
          },
          "instruction": {
            "type": "string",
            "description": "The action or prompt for this step, described in natural language"
          },
          "role": {
            "type": "string",
            "description": "If applicable, the role who performs this step (e.g., 'assistant' or 'user')"
          }
        }
      }
    }
  }
}
```

```

    "next": {
      "type": "string",
      "description": "ID of the next step (for non-linear flows or branching). If omitted, assume sequential next."
    }
  }
},
"end_condition": {
  "type": "string",
  "description": "Condition or criteria that ends the process (if applicable, e.g., 'user is satisfied' or 'all questions answered')"
}
}
}

```

This schema defines how an instruction pattern is structured. Each pattern has an ID, a textual purpose, and a list of steps. Each step carries an `instruction` (what the LLM should do at that step) and optionally a `role` (to clarify if the LLM is speaking or the user is expected to respond). A `next` field allows branching or jumps (e.g., in a quiz, a step might branch to a different next step depending on correctness; such logic could be handled by the orchestrator based on user input). The schema is kept generic; it does not enforce specific content of instructions, just the structure. This allows everything from linear workflows to simple branching logic to be expressed. An **example** following this schema is provided in **Boilerplate Examples** below (`example_process.json`).

4. `qa_rules.schema.json` – Schema for quality assurance (QA) rules definitions (for files like `checklists/diagram_quality.yaml` if converted to JSON, or any `qa_rules.json` if used):

```

{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "QA Rules Schema",
  "type": "object",
  "required": ["rules"],
  "properties": {
    "rules": {
      "type": "array",
      "description": "List of QA validation rules",
      "items": {
        "type": "object",
        "required": ["id", "description"],
        "properties": {
          "id": {
            "type": "string",
            "description": "Rule identifier"
          },
          "description": {

```

```

        "type": "string",
        "description": "What the rule checks for or enforces"
    },
    "severity": {
        "type": "string",
        "description": "Severity level if rule is violated (e.g., 'error',
'warning')"
    },
    "pattern": {
        "type": "string",
        "description": "Optional regex or pattern to detect the issue in
text (for automated checking)"
    },
    "suggestion": {
        "type": "string",
        "description": "Optional suggestion on how to fix a violation"
    }
  }
}
}
}
}
}

```

This schema captures how QA rules can be structured. Each rule has an `id`, a human-readable `description` of the check, a `severity` level, and optionally a `pattern` (which could be a regex or some identifier for programmatic checks) and a `suggestion` for corrections. For example, a rule might be: `id = "DiagramTitle"`, `description = "Every diagram must have a title node"`, `severity = "error"`, `pattern =` some regex to detect a title line, `suggestion = "Add a title line at the top of the diagram"`. The actual QA checks can be implemented by reading these rules and applying them to the LLM's output (either via code or by prompting the LLM to self-verify). The `checklists/diagram_quality.yaml` provided in the examples can be thought of as conforming to this schema (in YAML form, which is equivalent to JSON).

Note: All schemas above should be included in the bundle (e.g., in a `schemas/` directory) so that any tool or developer using the bundle can validate the files. This is important for maintaining a **contract**: it ensures the structure of files remains consistent as others adopt or contribute to the bundle.

RAG Manifest (index.json)

Below is an example `rag/index.json` manifest, illustrating how the RAG content might be listed. This follows the `rag_manifest.schema.json`:

```

{
  "bundle_name": "OpenRAG_Demo",
  "version": "1.0.0",
  "schema_version": "1.0",
  "description": "Example RAG bundle containing company policies and reference

```

```

articles.",
  "documents": [
    {
      "id": "policy_handbook",
      "file": "data/company_policy_handbook.md",
      "title": "Company Policy Handbook 2025",
      "topics": ["HR", "Policies"],
      "checksum": "sha256-3d2e4798..."
    },
    {
      "id": "tech_article_1",
      "file": "data/AI_research_trends.md",
      "title": "Research Trends in AI (2024)",
      "topics": ["AI", "Research"],
      "checksum": "sha256-a1b2c3d4..."
    },
    {
      "id": "tech_article_2",
      "file": "data/NLP_survey.txt",
      "title": "Survey of NLP Techniques",
      "topics": ["NLP", "Survey"],
      "checksum": "sha256-09f8e7d6..."
    }
  ]
}

```

Explanation: This manifest declares a bundle named "OpenRAG_Demo" at version 1.0.0. It lists three document files, each with an `id` used to reference the content, the file path, a title, some topic tags, and a checksum. In a real scenario, the `checksum` would be a full SHA-256 (or similar) hash string of the file content for verifying integrity. The presence of topics allows the retrieval system or LLM to filter or reason about which documents might be relevant (for instance, the manifest could be loaded into memory and searched by topic if needed, before resorting to full-text retrieval). This manifest is intended to be read by the application; the LLM itself typically doesn't read JSON directly (unless you decide to prompt it with this content, which is usually unnecessary). Instead, the app uses it to know what data is available and to validate that the files are present and untampered.

To adopt this in an open system: simply update the `documents` array with your own files. The manifest ensures anyone using the bundle knows exactly what's included and can verify it quickly. If the bundle is updated (say new documents added), the `version` should be bumped accordingly and new entries added here.

RIS Manifest (index.json)

Below is an example `ris/index.json` manifest, following the `ris_manifest.schema.json` structure:

```

{
  "bundle_name": "OpenRIS_Demo",
  "version": "1.0.0",
  "schema_version": "1.0",
  "patterns": [
    {
      "id": "interview_flow",
      "file": "patterns/example_process.json",
      "purpose": "Guided interview flow with user on a given topic"
    },
    {
      "id": "text_revision",
      "file": "patterns/revision_process.json",
      "purpose": "Multi-step text revision process (proofreading & editing)"
    }
  ],
  "prompts": [
    {
      "file": "prompts/interview_flow.md",
      "pattern_id": "interview_flow",
      "description": "Prompt scaffold for interview pattern"
    },
    {
      "file": "prompts/revision_flow.md",
      "pattern_id": "text_revision",
      "description": "Prompt scaffold for text revision pattern"
    },
    {
      "file": "prompts/narrative_frame.md",
      "pattern_id": null,
      "description": "General narrative writing framework prompt"
    }
  ],
  "templates": [
    {
      "file": "templates/swimlane_skeleton.mmd",
      "type": "mermaid",
      "description": "Mermaid flowchart skeleton for swimlane diagram"
    },
    {
      "file": "templates/minimal.bpmn",
      "type": "bpmn",
      "description": "Minimal BPMN process diagram template"
    },
    {
      "file": "templates/basic_sequence.puml",
      "type": "plantuml",

```

```

    "description": "Basic sequence diagram template in PlantUML"
  },
],
"checklists": [
  {
    "file": "checklists/diagram_quality.yaml",
    "scope": "diagram",
    "description": "Quality criteria for generated diagrams"
  }
],
"behavior": [
  {
    "file": "behavior/outputs.yaml",
    "scope": "output_format",
    "description": "Output formatting and style rules"
  }
]
}

```

Explanation: This manifest (OpenRIS_Demo v1.0.0) lists two patterns and their corresponding prompt templates, a narrative prompt that stands alone (no JSON pattern, perhaps it's a simpler guidance prompt), three templates for diagrams, one checklist, and one behavior config file. Each entry has a brief description to clarify its use.

For example, an application reading this manifest knows: - There is an **"interview_flow"** pattern with a JSON file and a matching prompt. When needed (likely triggered by a user request for an interview), these should be loaded. - There is a **"text_revision"** pattern for editing text, with its prompt. - A **"narrative_frame"** prompt exists that isn't tied to a JSON pattern. That could be used whenever a user requests a story or narrative structure. (In this case `pattern_id` is null or omitted, meaning it's a standalone prompt). - The **templates** section provides files that can be used if the output requires a diagram. For instance, if an answer needs a swimlane diagram, the system can fetch `swimlane_skeleton.mmd` as a starting point. The type field indicates how to interpret it (e.g., to feed into a Mermaid renderer or to ensure the LLM knows what syntax to use). - The **checklists** section indicates there is a YAML defining diagram quality rules. The `scope` "diagram" suggests these rules apply to any diagram output. - The **behavior** section points to an `outputs.yaml` that likely contains general output preferences.

In an open implementation, you might have code that loads `ris/index.json` at startup and uses it to configure the LLM's behavior. For example, one could automatically create a dictionary of pattern->prompt, so that when a pattern is to be used, the right prompt template is readily available. Or, if building a more autonomous agent, the agent could inspect this manifest to decide which patterns might apply to a given task (though such dynamic selection by the model itself would be advanced). At minimum, the manifest serves as documentation for developers and a sanity check (e.g., if a file listed here is missing, the system can warn the user that the bundle is incomplete).

Boilerplate Examples

The bundle comes with several boilerplate files to illustrate how to define processes, prompts, templates, and rules. All these examples are minimal yet copy-paste ready. They can be used as starting points for developing more complex logic and content.

File: `patterns/example_process.json` - This JSON defines a simple interview process pattern, following the `process_pattern.schema.json`. It guides an LLM through an interview-style interaction with a user on a given topic:

```
{
  "pattern_id": "interview_flow",
  "purpose": "Conduct an interview with a user about a specified topic.",
  "steps": [
    {
      "id": "step1",
      "instruction": "Greet the interviewee and introduce the interview topic.",
      "role": "assistant"
    },
    {
      "id": "step2",
      "instruction": "Ask the first open-ended question about [[Topic]].",
      "role": "assistant"
    },
    {
      "id": "step3",
      "instruction": "(Wait for user to respond to the question.)",
      "role": "user"
    },
    {
      "id": "step4",
      "instruction": "Acknowledge the answer and ask a follow-up question digging deeper into [[Topic]].",
      "role": "assistant"
    },
    {
      "id": "step5",
      "instruction": "(Wait for user to respond to the follow-up.)",
      "role": "user"
    },
    {
      "id": "step6",
      "instruction": "Summarize the discussion and thank the interviewee for their time.",
      "role": "assistant"
    }
  ]
}
```

```

  ],
  "end_condition": "Interview has concluded after the summary."
}

```

In this `example_process.json`, the pattern is identified as "interview_flow". The steps alternate between the assistant and (implicitly) the user. Steps with `role: "assistant"` indicate the LLM should produce an output (question or summary), while steps with `role: "user"` indicate the LLM should wait for a user reply (i.e., the conversation turn goes to the user). The placeholder `[[Topic]]` in the instructions signifies a slot that should be filled with the actual topic of the interview. The orchestration code using this pattern would replace `[[Topic]]` with the real topic (e.g., "climate change") in the prompt. This JSON doesn't directly execute on the LLM; instead, it's a blueprint that the system or the LLM's chain-of-thought can follow. For instance, an orchestrator can loop through these steps, prompting the LLM when `role` is "assistant", and collecting user input when `role` is "user". This ensures a structured interview. If one wanted a branching logic (not shown here), you could have a `next` field. But here it's linear, and the `end_condition` simply notes when to stop.

File: `checklists/diagram_quality.yaml` – A YAML checklist of quality rules for diagrams produced by the LLM (e.g., Mermaid or PlantUML diagrams):

```

rules:
  - id: "D1"
    description: "Every diagram must include a title or caption indicating its
subject."
    severity: "error"
  - id: "D2"
    description: "Each swimlane or participant should be clearly labeled (no
unnamed lanes)."
    severity: "warning"
  - id: "D3"
    description: "Diagram syntax should be valid (no syntax errors when
rendering)."
    severity: "error"
  - id: "D4"
    description: "Text in the diagram should be legible (avoid overly long
labels or overlapping text)."
    severity: "warning"

```

This checklist defines four rules (D1–D4). We use `severity` levels "error" for critical issues that must be fixed, and "warning" for less critical suggestions. In practice, an automated validator might use this file by reading each rule: for D1, it might check the diagram source text for the presence of a title (depending on diagram format, e.g., in Mermaid maybe a top-level text or a commented title). D3 suggests running the diagram through a parser or renderer to catch syntax errors. These rules can be enforced in a pipeline: after the LLM generates a diagram, the system can verify each rule. For example, it could detect if a Mermaid diagram lacks a `:::` style title section or a top node labeled "title". Or an LLM could be prompted to self-check: "Does the diagram have a title? If not, add one." Using a YAML like this for rules is similar to how

guardrail frameworks allow defining output constraints and checks ⁴. The advantage of keeping it in a file is that we can update or add rules without modifying code. Developers adopting this bundle would customize `diagram_quality.yaml` to their preferences (maybe add rules, e.g., "D5: All critical steps must be present in the diagram." etc.).

File: `behavior/outputs.yaml` – A YAML configuration for output behavior and format preferences. This example sets global rules for how the LLM should format answers:

```
format_preferences:
  primary: "markdown"          # Primary format for text output
  code_blocks: "fenced"        # Use fenced code blocks for any code or multiline
                                output
  include_citations: false     # (Example) Whether to include citations by default

style_preferences:
  tone: "formal"               # Use a formal tone in responses
  allow_first_person: false    # Avoid using "I" or personal POV in answers

length_limits:
  max_tokens: 2048             # Suggest not exceeding ~2048 tokens in an answer
  max_sections: 4              # If splitting answer into sections, limit number
                                of sections

fallback_strategy:
  on_failure: "graceful_exit"  # If the task fails or is unsafe, respond with a
                                polite refusal/apology
  on_ambiguity: "ask_clarification" # If user query is unclear, ask a
                                    clarifying question
```

In this `outputs.yaml`, we define a few sections: - **format_preferences**: e.g., use Markdown for output (common for readable formatting), and ensure code or diagrams are in fenced code blocks (which is good practice for readability). Also a hypothetical flag for citations. - **style_preferences**: e.g., always maintain a formal tone, and avoid first person to keep answers objective (this could help in contexts like reports or knowledge-base style answers). - **length_limits**: some soft limits on answer size or structure complexity. - **fallback_strategy**: how the LLM should respond if it cannot comply (maybe the instructions in system prompt will use these flags to decide whether to ask clarifying questions or to refuse certain requests).

These settings would typically be enforced by incorporating them into the system prompt or by logic in the application. For instance, the system prompt might be constructed to say "You should respond formally and avoid first person. If the user request is unclear, ask for clarification. If the request cannot be answered safely or goes beyond scope, respond with a polite refusal." The YAML provides a structured way to maintain these guidelines. Developers can modify `outputs.yaml` to adjust the LLM's behavior without hardcoding changes. This could also feed into the QA checks (e.g., a QA rule could check that the output contained no first-person pronouns if `allow_first_person: false`).

File: `templates/swimlane_skeleton.mmd` - A Mermaid diagram template for a simple swimlane flowchart:

```
%% Swimlane diagram skeleton
flowchart LR
    subgraph Lane_A [Lane A]
        A1[Start] --> A2[...]
    end
    subgraph Lane_B [Lane B]
        B1[...] --> B2[End]
    end
    A2 --> B1
```

This Mermaid code defines a basic structure with two lanes (Lane A and Lane B). Lane A has a start node leading to an intermediate node `A2[...]`, Lane B has an intermediate `B1[...]` leading to an end node. The `A2 --> B1` line shows a cross-lane transition. The idea is that the LLM can use this as a starting point when asked to produce a swimlane diagram; it just has to replace `[...]` with actual steps relevant to the query and possibly add more nodes or lanes as needed. The comment line `%% Swimlane diagram skeleton` is ignored by Mermaid but helps a developer know what the template is.

Using templates like this ensures that the LLM outputs diagrams that conform to a correct syntax structure. Instead of expecting the model to produce a correct Mermaid from scratch every time, we provide a scaffold. The model (or more likely, the orchestrating system) could choose the `swimlane_skeleton.mmd` when a question asks for a process flow with multiple actors. It can then fill in the steps or rename lanes appropriately. This increases reliability of the output.

File: `templates/minimal.bpmn` - A minimal BPMN (Business Process Model and Notation) XML template:

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.omg.org/spec/BPMN/20100524/MODEL
BPMN20.xsd"
    id="Definitions_1" targetNamespace="http://example.com/bpmn">
    <process id="Process_1" isExecutable="false" name="BasicProcess">
        <startEvent id="StartEvent_1" name="Start"/>
        <endEvent id="EndEvent_1" name="End"/>
    </process>
</definitions>
```

This is a very basic BPMN file: a definitions wrapper with a single process that has a start event and an end event. BPMN is an XML format, and it can get very complex, but this provides the skeleton. If the LLM needs to output a BPMN diagram (for instance, if the user requests a formal BPMN representation of a workflow), the model could use this template as a starting point, then insert tasks, gateways, etc., between the start

and end as needed, and adjust the `name` attributes accordingly. In a practical system, the LLM might not spontaneously write XML unless guided by a prompt. So you might have a prompt scaffold that says, “Use the following BPMN XML template and insert steps corresponding to the described process.” Then provide this minimal template for the model to elaborate. By including the proper namespace and schema location in the template, we also ensure that any BPMN tool can open the resulting file without errors. This template is vendor-neutral (BPMN is an open standard) and does not rely on any specific software.

File: `templates/basic_sequence.puml` – A basic PlantUML sequence diagram template:

```
@startuml
actor User
participant System
User -> System: Request
System -> User: Response
@enduml
```

This PlantUML code describes a simple sequence diagram with two entities: a User and a System, where the User sends a request and the System responds. As a template, this gives the structure of a sequence diagram. An LLM can take this and expand on it: e.g., add more participants or more messages as needed. PlantUML is a text-based UML specification that can be rendered by various tools (and it’s open source). The advantage of providing this is similar to the Mermaid case: to ensure correct syntax (`@startuml` ... `@enduml` wrapper, correct arrow notation `->`, etc.). If a user asks, “Illustrate the interaction between X, Y, and Z in a sequence diagram,” the model can start from this basic template, add actors for X, Y, Z, and multiple interactions.

Because sequence diagrams follow a predictable pattern, a template ensures the model doesn’t forget the start/end or other boilerplate. Also, if combined with QA rules, you can check that any `@startuml` has a matching `@enduml` (to catch incomplete diagrams).

File: `prompts/interview_flow.md` – A prompt scaffold for conducting an interview (to be used with the interview pattern). This Markdown content would be injected into the LLM’s system or assistant prompt to guide its behavior:

```
You are an interviewer tasked with conducting an interview about [[Topic]].
Follow the interview procedure strictly:

1. Introduction: Greet the interviewee and briefly introduce the topic of [[Topic]].
2. Questioning: Ask an open-ended question on [[Topic]]. Allow the interviewee to respond in detail.
3. Follow-up: Based on the interviewee's answer, ask a relevant follow-up question to dig deeper into [[Topic]].
4. Continuation: Continue this question-answer cycle, ensuring each question remains on the subject of [[Topic]] and encourages thoughtful responses.
5. Conclusion: Conclude the interview by summarizing the key points
```

discussed and thank the interviewee for their time.

Adhere to this structure and do not stray from the topic. Remain neutral and professional throughout the interview.

In this scaffold, `[[Topic]]` is a placeholder to be substituted with the actual topic. The prompt explicitly instructs the model on the structure (introduction, questioning, follow-up, etc.). The numbered list provides a clear sequence that aligns with the `example_process.json` steps (though not one-to-one, it conceptually matches introduction -> Q&A cycle -> conclusion). By enumerating the steps in the prompt, we leverage the LLM's strength in following ordered instructions. The tone is set to "neutral and professional," which complements any style rules from `outputs.yaml`.

A developer would take this file, replace `[[Topic]]` with, say, "climate change" if the user wanted an interview about climate change, and then include it at the appropriate place in the LLM conversation (likely as a system message or prefix to the assistant's response). The LLM should then conduct the interview according to the outline. This separation means the user's actual question could be something simple like, "Interview me about climate change." The system recognizes the need for the interview pattern and uses this prompt to instruct the model, rather than relying on the model to come up with an interview structure on its own every time.

File: `prompts/revision_flow.md` – A prompt scaffold for a text revision process (e.g., proofreading/editing):

You are an expert copyeditor. Your task is to refine and improve a given text.

Procedure:

1. **Read the original text carefully.** Identify any grammatical errors, unclear phrasing, or inconsistencies.
2. **List Issues:** If there are multiple issues, list them clearly (grammar mistakes, style improvements, etc.).
3. **Suggest Corrections:** Provide a correction or improvement for each issue identified. Be specific and concise.
4. **Revised Text:** Present a fully revised version of the original text, incorporating all the improvements you suggested.
5. **Explanation (optional):** If helpful, briefly explain major changes made (e.g., why a certain sentence was rephrased for clarity).

Follow the above steps closely. Ensure the final revised text is clear, coherent, and free of errors. Do not change the meaning of the original text.

This prompt instructs the LLM on how to perform a multi-step revision of a text. It's formatted with bold headers for each stage. The LLM, when given this scaffold plus the text to revise, should ideally list issues, suggest corrections, and then produce the improved text. By structuring it, we avoid a scenario where the model either just rewrites the text without explanation or only lists problems but doesn't fix them. It aligns with an RIS pattern (if one were defined for revision). Even if we don't have a JSON pattern for it in the example, the prompt itself is sufficient to guide the process. A developer can insert the original text after

this prompt (e.g., "Original Text: ...") or as part of the user prompt, and the model will follow these steps. This structured approach also makes it easier to apply QA: for instance, we can check that the model's answer indeed has a list of issues and a revised text. If the model skips listing issues, we know it deviated from instructions.

File: `prompts/narrative_frame.md` – A prompt scaffold to frame a narrative (storytelling) output:

You are a storyteller crafting a narrative. Use the following structure in your story:

- ****Introduction:**** Set the scene and introduce the main character(s) and context.
- ****Rising Action:**** Develop the story by introducing a challenge or conflict. Build suspense or interest.
- ****Climax:**** Describe the most intense point of the story, where the main conflict reaches a turning point.
- ****Falling Action:**** After the climax, show the consequences and how the conflict moves toward resolution.
- ****Conclusion:**** Resolve the conflict and conclude the story, providing a clear ending or moral.

Make sure the narrative flows logically from one part to the next. The tone should be engaging and appropriate for the content of the story. Avoid irrelevant tangents and maintain consistency in the plot and characters.

This scaffold breaks a typical narrative arc into labeled sections. It tells the LLM exactly what to do in each part of the story. This is extremely useful when a user requests a structured story or when we want the story to have a beginning, middle, and end. The bullet points with bold section names act as guideposts. A clever model might even include those section headings in its output (which might or might not be desired), but the instruction “use the following structure” implies it should ensure the content follows that flow. If we didn’t want the headings explicitly in the output, we could tweak the wording to “Ensure your story has: an introduction, rising action, climax, etc., though you need not label them.” In any case, this prompt helps maintain coherence. Without it, the model might tell a story that has an introduction and conclusion but muddles the middle. This frame ensures even an open-model knows to include a climax and resolution, which improves quality.

These prompt files demonstrate how RIS can provide *content blueprints* for the LLM. They are **vendor-neutral** (no product-specific instructions) and focus on logical structure, so any modern LLM should benefit from them. Users building an open system can extend these scaffolds or create new ones as needed, and include them in the `prompts/` directory with appropriate entries in the manifest.

QA.md (Quality Assurance Guidelines)

Quality assurance is crucial in a RAG+RIS system to ensure the outputs are correct, safe, and in the desired format. The **QA.md** document explains how to apply QA rules and validate LLM outputs using the provided checklists and additional strategies:

Quality Assurance (QA) for LLM Outputs

This document describes how to enforce and verify quality rules for the content generated by the LLM, using the rules in the RIS bundle (e.g., checklists like ``diagram_quality.yaml``) and general best practices.

QA Process Overview

1. **Automated Structure Validation**: Use schema and format validators to ensure outputs are well-formed. For example:
 - If the LLM returns JSON or XML (for a diagram or structured data), run it through a JSON Schema validator or XML parser.
 - If the output is supposed to follow a specific template (like a diagram syntax), use a linter or parser for that format (e.g., a Mermaid syntax checker).
2. **Rule-based Checks**: Refer to the rules defined in ``checklists/`` YAML files:
 - Each rule has an ``id``, ``description``, and severity. Implement automated checks for each rule where feasible. For instance, for rule **D3** ("Diagram syntax should be valid"), attempt to render or parse the diagram. For **D1** (diagram title), search the text for the presence of a title line.
 - The ``pattern`` field in ``qa_rules.schema.json`` can be used if provided. For example, a rule might include a regex pattern to detect profanity, disallowed phrases, or missing elements. Use these patterns to scan the output.
 - Mark any violations. For **error** severity, the output is not acceptable and the LLM should be prompted to correct it (or the system should discard/re-do the response). For **warning**, note the issue for review or possible revision.
3. **Content Consistency and Factual Accuracy**:
 - Cross-verify any factual statements against the source content. In a RAG setup, if the LLM cites something from a document, ensure that claim is actually supported by that document. This can be manual or automated by searching the retrieved context for key phrases.
 - If the system has access to a separate verification model or tool, it could ask a second model to fact-check the first model's answer against the given context.
4. **Adherence to Instructions**:
 - Ensure the LLM followed the pattern/prompt. If the chosen pattern was "interview_flow", check that the conversation in the output indeed has a greeting, questions, follow-ups, and a conclusion. If a step was skipped, that's a QA failure.
 - The system can automate this by expecting certain keywords or structures in the output. For example, after an interview, the last assistant message should contain a thank you (as per the prompt scaffold). If not present, the model may have deviated.

- For multi-step processes (like revision), check that each requested component (list of issues, suggestions, revised text) is present.

5. ****Style and Safety Checks****:

- Use the preferences in ``behavior/outputs.yaml`` to enforce style. For instance, if ``allow_first_person: false``, scan for " I " or "my" in the output from the assistant and flag if found outside of quotes/examples.
- Check for toxic or biased language. While our bundle doesn't include a specific toxicity list, it's good practice to run outputs through a filter (or a classification model) to catch unsafe content.
- Ensure no internal instructions or sensitive info leaked. The LLM should not output system prompts or any content from RIS that wasn't meant for the user. In practice, this means verifying that placeholders like `[[Topic]]` are replaced and do not appear literally, and that the model didn't expose something like the pattern `JSON content`.

Using the QA Checklist Programmatically

The file ``checklists/diagram_quality.yaml`` can be converted to JSON and validated against ``qa_rules.schema.json`` for a structured interpretation. For example, a simple Python script could load the YAML as data and iterate rules:

```
```python
import yaml, re
rules = yaml.safe_load(open('checklists/diagram_quality.yaml'))
output = open('llm_output.txt').read()

issues = []
for rule in rules['rules']:
 if rule.get('pattern'):
 # If a regex pattern is provided, use it to check output
 if re.search(rule['pattern'], output) is None:
 issues.append(f"{rule['id']}: {rule['description']}")
(This pseudocode assumes pattern is written such that a *failure* to find a
match means an issue.
Other rules without 'pattern' would require custom logic or manual review.)
```

After such checks, the script could print issues or even attempt an automatic fix (e.g., if the title is missing, prompt the LLM to add one). Modern guardrail tools allow YAML-based validators with built-in checks like regex, length, semantic similarity, etc., and can auto-retry generation <sup>4</sup>. Adopting those patterns, our QA ensures outputs adhere to expected schemas and content guidelines.

## Human Review and Iteration

Even with automated QA, human oversight is advised for critical applications: - **Spot-check** a sample of outputs regularly, comparing them against the source documents and intended format. - Update the rules in `checklists/` as new failure modes are discovered. - Use version control for the RIS bundle; if the LLM consistently fails a certain QA check, consider updating the prompt or pattern to address it.

By following these QA steps, you maintain a **feedback loop**: model outputs are validated, errors are caught and corrected, and the instructions (RIS) or data (RAG) can be refined over time to improve quality. This disciplined approach is key to reliable, safe LLM deployments.

```
Summary: The QA.md instructs developers how to systematically verify LLM outputs. It leverages the structured rules we've defined (like those in `diagram_quality.yaml`). The approach is similar to using guardrails or validators to enforce output correctness 4. We emphasize that severe issues should trigger a re-generation or correction by the model (possibly by feeding back an error message or an adjusted prompt), whereas minor issues can be flagged. Over time, as the LLM is tuned or the prompts are adjusted, these QA checks help measure progress (fewer violations over time) and ensure that even as the content and logic bundles evolve, the outputs remain consistent with the requirements.
```

### **## Security\_Integrity.md (Signing, Checksums, Versioning)**

Security and integrity of the RAG/RIS bundles are paramount, especially if these bundles are shared or used in sensitive contexts. The **\*\*Security\_Integrity.md\*\*** covers how to trust and verify the bundle:

```
```markdown
```

Security and Integrity of RAG/RIS Bundles

To prevent unauthorized modifications and ensure the LLM uses trustworthy instructions and data, we implement several security measures:

File Checksums

Every content file in the bundle is accompanied by a checksum (hash value). In the manifest files (`rag/index.json` and `ris/index.json`), the `checksum` field (typically SHA-256) is provided for each entry:

- Before using a file at runtime, compute its SHA-256 hash and compare it to the manifest. If they differ, the file may have been altered or corrupted. In such a case, the system should ****reject**** the bundle or at least skip that file and log an integrity error.
- These hashes ensure that even if the bundle is stored or transmitted, any bit flips or tampering can be detected reliably (SHA-256 has an extremely low collision probability).

Developers should use standard tools (e.g., `shasum` on UNIX or a Python hashing library) to verify checksums as part of the startup sequence of the application.

Digital Signing

While checksums protect against accidental or malicious changes, they assume the manifest itself is trusted. For stronger guarantees:

- The bundle provider (e.g., the team curating RAG/RIS content) can sign the `index.json` files using a public/private key pair. For example, generate a signature file like `index.json.sig` using an RSA or Ed25519 key.
- Consumers of the bundle can verify this signature using the provider's public key. This assures that the manifest (and thereby the listed content) indeed comes from the expected source and hasn't been swapped out by an attacker.
- Signing the entire bundle as an archive: Alternatively, you can distribute the RAG/RIS bundle as a ZIP or TAR archive. By signing the archive or providing a signed manifest of all file hashes, you simplify verification (one signature to check instead of many). This approach is similar to how software distributions are signed.

It's recommended to use existing standards (e.g., PGP signatures or code signing certificates) so that verification can be automated with known tools.

Version Pinning and Compatibility

Each bundle has a version (following semantic versioning). For security and consistency:

- The LLM application can be configured to accept only a specific bundle version or range of versions that are known to be compatible. If an unexpected version is loaded, the system might log a warning or refuse to start, preventing unknown changes.
- If bundles are downloaded or updated dynamically, use the version and possibly an allowed list of checksums to ensure you got exactly the intended file (this can thwart supply chain attacks where an attacker might try to trick the system into loading a malicious bundle).

Isolation of Instructions vs Data

As a security principle, **instructions (logic) and data (content) are stored separately** (RIS vs RAG). This design, beyond organizational benefits, limits the impact of a potential breach:

- If someone maliciously edits a RAG document to include a hidden instruction (e.g., "Ignore previous directives..."), a properly designed system would treat it as data, not as an executable command, thanks to the RIS enforcement. The model's prompt should delineate what is background data vs. what are instructions.
- We recommend implementing the prompt assembly such that any content from RAG is clearly quoted or tagged as user-provided/reference content, whereas RIS instructions are in a protected system section of the prompt. This makes the separation clear to the model (many LLMs respect system-vs-user prompt distinctions).

Secure Updates

When updating either bundle:

- Use a secure channel to distribute the update (e.g., a verified repository or

server).

- Re-verify all checksums post-update.
- If partial updates are allowed (like adding one document), update the manifest carefully and increment the version. A diff of manifests can be code-reviewed to see exactly what changed.

Audit Logging

In a production environment, log events such as:

- Bundle load events (which version loaded, were signatures verified?).
- Any checksum mismatches or signature failures.
- Any QA rule violations (as they might indicate either a model trying to do something odd or possibly someone tampering with prompts).

By reviewing logs, you can detect anomalies. For example, if suddenly a lot of outputs fail QA with a specific pattern, it could hint that either the model was changed or someone altered the instructions.

Dependency Considerations

This bundle is vendor-neutral and doesn't rely on proprietary binaries, but if it references external tools (for rendering diagrams, etc.), ensure those tools are also verified. E.g., if using an open-source Mermaid renderer to visualize diagrams, use a trusted version to avoid injection vulnerabilities in rendering.

Summary

Through checksums, digital signatures, strict version control, and careful separation of roles, we maintain the integrity of the RAG/RIS bundles. These measures analogously follow practices in traditional software security (like code vs data separation in memory ⁵, and signed software releases) to make sure our LLM operates on **authentic instructions and data** at all times.

Key Points from Security_Integrity.md: We ensure each file's integrity (via checksums), authenticity of the manifest (via signatures), and correct versioning. The note about instruction vs data echoes the earlier academic point – similar to how CPU architectures mark data vs code ⁵, we conceptually do the same with RAG vs RIS. This reduces certain attack surfaces (someone slipping a malicious instruction in a data file, for instance). Additionally, the emphasis on semantic versioning and not loading unknown versions ensures that even if someone tries to drop in a malicious older/newer version of a bundle, the system can catch it. All of this is achieved with open, standard tools (no proprietary service needed to compute a SHA-256 or verify a signature).

Maintenance.md (SemVer Policies and Archiving Strategy)

This document **Maintenance.md** outlines how to manage the evolution of the RAG and RIS bundles over time, using semantic versioning and archiving old versions for traceability:

Maintenance and Versioning Strategy

To ensure the RAG/RIS bundles remain reliable as they evolve, we adhere to semantic versioning and maintain an archive of changes.

Semantic Versioning Policy

We follow **Semantic Versioning 2.0.0** ³ for the bundle versions (as indicated in each `index.json` manifest):

- **MAJOR version** (X.y.z): incremented for incompatible changes. This includes removing or renaming fields in schemas, removing significant content, or changes that require updates in the consuming application or prompt logic. A jump in MAJOR means adoptors need to review integration points.
- **MINOR version** (x.Y.z): incremented for backward-compatible additions. E.g., adding new documents to RAG or adding new patterns/templates to RIS that do not break existing ones. Consumers can upgrade to a new minor version and expect existing functionality to continue working, though new features are available.
- **PATCH version** (x.y.Z): incremented for minor fixes, such as correcting a typo in a prompt, tweaking a QA rule, or adding a small content update that doesn't affect overall usage. Patches should not affect the LLM's behavior significantly (except to fix errors).

Once a version is released, its contents **must not change** ⁶. Any modification creates a new version. This immutability ensures consistency: if someone rolls back to a previous version, they get the exact same files that were originally released under that version.

Deprecation and Compatibility

When introducing a change that might affect how the LLM or system behaves:

- Clearly **deprecate** old patterns or prompts in documentation. For example, if a pattern is replaced by a more effective one, mark the old pattern as deprecated (perhaps move it to a `legacy/` folder or add a note in its description). Continue to carry it through the current MAJOR version to not break existing users, if feasible.
- If a certain manifest field is to be phased out (rare, but e.g., if we decided to stop using `topics` in RAG documents), first mark it as deprecated (keep it but no longer rely on it), and remove it only in the next major release.

For the LLM runtime, ensure that any critical changes are flagged:

- If a new version requires an update in the prompt assembly logic or retrieval logic, the README and Adoption_Playbook should be updated to call that out.
- Maintain a **changelog** (in this Maintenance.md or a separate CHANGELOG.md) that summarizes changes by version. This helps users quickly see what changed.

Archiving Strategy

All released versions of the bundles should be preserved (read-only) in an archive:

- Use a version control system (e.g., git tags or branches for each release) so that one can fetch the state of the repository at any released version.
- Alternatively, keep a directory of released bundle zips with the version in the filename (e.g., ``OpenRAG_Demo_1.0.0.zip``, ``OpenRAG_Demo_1.1.0.zip``, etc.), ideally with accompanying signatures/checksums as mentioned in `Security_Integrity.md`.
- When the MAJOR version increments, consider creating a new branch or folder for that series. E.g., ``v2.x`` branch for all 2.*.* releases, separate from the ``v1.x`` history, to avoid confusion.

Archiving is important not just for rollback, but also for auditing. If a particular answer or behavior was observed with an older bundle, having that exact bundle allows analysis of what instructions or data led to it.

Community Contributions and Curation

If this is an open bundle that others can contribute to:

- Use a review process for merging changes. Given the potential impact on AI behavior, treat changes to RIS (prompts/patterns) with the same rigor as code changes in software.
- Update the version number as part of any change merge. Automate this if possible (e.g., require that a PR includes a version bump if files in certain directories changed).
- Validate the bundle on each change: run schema validation and perhaps a set of QA tests (some of the test cases from `Adoption_Playbook.md`) to ensure nothing breaks.

Long-term Maintenance

As LLMs improve, some patterns or prompts may become obsolete (the model might handle things more autonomously). Nonetheless, maintaining them causes little harm; they can still act as guardrails. Only remove features in a major version upgrade after careful consideration.

- Aim to keep the bundle lean but comprehensive. Removing redundant or ineffective items can simplify the model's job, but do it in a controlled manner.
- Monitor usage: if certain RAG documents are never used or certain RIS patterns never trigger, that might indicate they can be pruned or need revision.

By following these maintenance practices, the bundle will remain a ****living resource**** that can grow and adapt without sacrificing stability or backward compatibility.

In summary, **Maintenance.md** ensures there's a clear policy for version numbers (tying into SemVer best practices ³) and an emphasis on not modifying released versions (to guarantee reproducibility ⁷). The archiving strategy is about keeping historical copies, which is crucial for an open system where multiple parties may rely on the bundle. We also briefly mention contribution if it's open source – even if not explicitly asked, it's part of maintenance to consider how changes are integrated. The overarching goal is to avoid “breaking the chain”: an LLM system might be wired intricately with the prompts/data; uncoordinated changes could degrade performance or accuracy. The policies here help mitigate that.

Adoption_Playbook.md (Integration Guide and Test Cases)

The Adoption Playbook provides a step-by-step guide for integrating the RAG and RIS bundles into an LLM workflow, including an ASCII diagram of the flow and example test cases to validate the setup:

```
# Adoption Playbook for RAG + RIS Integration
```

```
This playbook walks through how to wire the Remote Instruction Stub (RIS) and Retrieval-Augmented Generation (RAG) components into your LLM pipeline. It also provides an ASCII diagram of the process flow and a set of test cases to ensure everything works as expected.
```

```
## Integration Steps
```

```
**1. Bundle Acquisition and Verification:**
```

```
- Obtain the RAG and RIS bundles (file structure as described). This could be via git clone, downloading a zip, etc.
- Verify integrity: run checksum and signature verification as per *Security_Integrity.md*. Make sure `index.json` files match their schemas (you can use a JSON Schema validator to catch any format issues early).
```

```
**2. Environment Setup:**
```

```
- Ensure you have an LLM available (local or via API) and an embedding model for retrieval. All should be open-source or self-hosted if avoiding proprietary tools.
- Install any necessary libraries for your programming environment (for example: a vector database or FAISS for retrieval, a JSON parser, YAML parser for reading configs, etc.).
```

```
**3. Load RAG Content into Retrieval System:**
```

```
- Read `rag/index.json`. For each document entry, load the file from `data/` and add it to your retrieval index. If using embeddings, generate embeddings for each document (unless you have a precomputed index in `embeddings/` to load).
- Store a mapping from document `id` to the actual content (or file reference) so you can quickly fetch content after retrieval.
- Ensure the vector index is saved or ready in memory for queries.
```

```
**4. Load RIS Logic and Prepare Prompts:**
```

- Read ``ris/index.json``. Load all patterns and prompts into memory (e.g., into dictionaries or classes in your code).
- You might create a mapping of pattern ID to its JSON structure (for logic decisions), and pattern ID to its prompt template text.
- Also load any templates (you can keep them as multi-line strings, to insert into outputs when needed) and the behavior config (to inform prompt construction or output post-processing).
- For each pattern, you could write a small handler or function. For example, an ``interview_flow`` handler that given a topic, will use the interview prompt and manage turn-taking with the user.
- Incorporate the behavior rules: e.g., if ``outputs.yaml`` says tone = formal, you prepend a note in the system prompt about maintaining a formal tone.

****5. Runtime Query Handling:****

- When a user query comes in, first determine if a special pattern applies. This can be done via simple keyword matching or a classifier. (E.g., if the query is "I'd like an interview about X", or "Can you revise this text: ...", those map to `interview_flow` or `text_revision` patterns.)
- If a pattern is triggered (say ``pattern_id = "interview_flow"``):
 - Retrieve the corresponding prompt scaffold (e.g., ``interview_flow.md``), fill in placeholders (like `[[Topic]] = X`).
 - If the pattern involves multiple steps, you have two options:
 1. ****Single LLM session approach:**** Craft a combined prompt that includes the scaffold and possibly instructs the model to conduct the whole sequence internally (this can work for shorter interactions or where the LLM plays both roles).
 2. ****Interactive loop approach:**** Feed the model step by step. For an interview, you prompt the LLM with step1 (greeting) and step2 (first question) together, get output, then wait for user input, then feed step4 (follow-up) etc., as per ``example_process.json``. This requires your application to manage turns.
 - Use the ``behavior/outputs.yaml`` settings: for instance, ensure the prompt format is Markdown if required, enforce any style instructions by adding them to the system prompt.
 - If no special pattern is needed (just a normal Q&A or general query):
 - Construct a default prompt (system message) that might include generic instructions (like "You are a helpful assistant." plus any style rules from ``outputs.yaml``).
 - Proceed to retrieval.
- ****Retrieval (RAG):**** For any user query (pattern or not), identify if external knowledge is needed. If the query is factual or requests info (e.g., "What is the company policy on remote work?"), perform these sub-steps:
 - Compute the query embedding and find top-K relevant documents from the RAG index.
 - Retrieve the content (exact paragraphs or sections) from those documents.
 - Attach this content to the prompt. Typically, you add a section like:


```
"\nRelevant information:\n* Document Title: ... (content snippet) ...".
```


- Make sure the content is clearly marked as context, not instructions. You might label it as "Background" or something in the prompt.

- Finally, send the assembled prompt (with instructions from RIS and context from RAG, plus the user question) to the LLM for generation of the answer.

****6. Post-Generation:****

- Take the LLM's output and run the QA checks (see QA.md). If any checks fail:

- You can loop back and correct the prompt or gently nudge the LLM to fix the output. For example, if a diagram is missing a title (violates rule D1), you can append a prompt: "Oops, the diagram is missing a title. Please add a title." and regenerate.

- Or use function calls/tool use if available in your LLM to fix formatting issues.

- If the output passes QA, present it to the user.

- Log the interactions if needed for future analysis (especially if something went wrong).

Throughout these steps, maintain clear ****separation****: RIS content (prompts, patterns) influence the model's behavior, while RAG content (retrieved data) influences the model's knowledge. The orchestration logic you implement ensures they merge at the right moment.

ASCII Flow Diagram

Below is an ASCII diagram illustrating how a user query flows through the RIS and RAG components to produce a final answer:

```
User Query | v +-----+ 1. Pattern Selection & Instruction Injection +-----+ | RIS
|-----> | | (Logic & 2. Retrieve relevant data (if needed) | LLM | | Prompt)
|-----> | (Model | +-----+ | Engine) | ^ (3) | | | +-----+ | (If needed: User
responds, loop via RIS) | | v | '-----+ +-----+ | | RAG | (4) Context | +--->|
(Knowledge|----> merge | Base) | | +-----+ v Answer to User
```

****Diagram Explanation:****

1. The ****RIS stage**** processes the user query to determine the instruction pattern. For example, if the query is "Interview me about climate change", RIS picks the interview pattern and prepares the interview prompt.
2. The query (possibly modified or augmented by RIS instructions) then may trigger a ****RAG retrieval**** if information is needed. In a standard Q&A, RIS might just be a default logic that always does retrieval. In an interview, maybe RAG isn't used at first (since it's just Qs from the interviewer), but if the interviewee asks a factual question back, the system could use RAG then.
3. Both the instructions from RIS (like the structured prompt) and data from RAG (retrieved context) are merged into the LLM's input. The LLM then generates a response following those instructions and using that data.

4. The diagram shows a potential loop: after the LLM responds (especially in multi-turn patterns), the user's answer might go back into RIS (the orchestrator) to decide the next step. This continues until the pattern concludes, then the final answer or outcome is delivered.

Thus, the flow is ****User -> RIS -> (maybe loop user/RIS) -> RAG -> LLM -> Answer****. In a simpler single-turn Q&A, it reduces to User -> RIS (choose default Q&A prompt) -> RAG (fetch docs) -> LLM -> Answer.

Example Test Cases

To validate the integration, use the following test scenarios:

****Test Case 1: Basic Factual Q&A****

User prompt: "What are the company's policies on remote work?"

****Expected behavior:**** The system uses a default QA pattern. The RAG retrieval finds the "company_policy_handbook.md" content about remote work. The LLM is prompted with that info plus the question.

Expected output: A concise answer explaining the remote work policy, grounded in the handbook (perhaps quoting it). No extraneous info. (Verify that if the policy says, e.g., "2 days remote allowed per week," the answer reflects that exactly.)

****Test Case 2: Interview Pattern****

User prompt: "Can you interview me about climate change?"

****Expected behavior:**** The system selects the `interview\_flow` pattern. It fills the scaffold with Topic = climate change. The LLM greets the user and asks a question about climate change. When the user (tester) provides an answer, the system (if in interactive mode) feeds that back in and prompts the LLM for a follow-up question, and so on.

Expected output: A series of turns where the assistant strictly follows the interview format: introduction, question, follow-up, conclusion. The conversation should stay on climate change. The final assistant message should summarize and thank the user. (Check that it did not break character or inject unrelated content.)

****Test Case 3: Revision Pattern****

User prompt: "Here is a draft: 'This is an exampel of a bad sentence.' Can you proofread it?"

****Expected behavior:**** The system triggers the `text\_revision` pattern (recognizing "proofread" or format). The `revision\_flow.md` prompt is used. The LLM should list issues (e.g., spelling of "exampel"), suggest corrections, then give a revised sentence.

Expected output: The assistant might produce:

1. Issues: "- Spelling error in 'exampel'."

2. Suggestions: "- Change 'exampel' to 'example'."

3. Revised Text: "This is an example of a bad sentence." (Though ideally, it might also improve "bad sentence" to something like "poorly constructed")

sentence" depending on instructions.)

Ensure the output format matches the scaffold (the steps are clearly delineated). If the model just corrected the sentence without explanation, that's a fail on following the procedure.

****Test Case 4: Narrative Frame****

User prompt: "Tell me a short story about a wise old tree."

****Expected behavior:**** This might not explicitly trigger a pattern unless we decide any "story" request uses `narrative\_frame.md`. Assuming we do that, the system applies the narrative scaffold.

Expected output: A story with a clear beginning, middle, and end. Check that the story indeed has an introduction (setting the scene, maybe introducing the tree), a conflict or challenge (perhaps the forest is in danger or the tree imparts wisdom), a climax (peak of action or emotion), and a resolution. The transitions should be smooth. If the output lacks a clear climax or conclusion, then the scaffold wasn't properly followed or enforced. Possibly iterate on prompt if needed.

****Test Case 5: Diagram Request****

User prompt: "Show me a sequence diagram of a user logging into a system."

****Expected behavior:**** The system might detect keywords "sequence diagram" and decide to use the `basic\_sequence.puml` template. It retrieves no external data (the scenario is generic), but it uses RIS logic: maybe a pattern or prompt that specifically instructs diagram creation. (We didn't explicitly have a pattern for this, but one could imagine a generic "diagram_creation" pattern).

The LLM should output PlantUML code, based on `basic\_sequence.puml`, with "User" and "System" as participants and appropriate messages for login (e.g., "User -> System: enter credentials", "System -> User: authentication result").

Expected output: Proper PlantUML syntax from `@startuml` to `@enduml` with the steps filled in. Use QA to validate it: check that it starts with `@startuml` and ends with `@enduml` (structure), that it has at least one arrow from User to System and back. The `diagram\_quality.yaml` doesn't directly cover PlantUML, but general rules like having a title might not apply here. Ensure no obvious mistakes (like missing enduml). If any, adjust the template or prompt and test again.

By running these test cases, you can confirm:

- The system correctly identifies when to use RIS patterns.
- RAG retrieval returns relevant info for factual queries.
- The combined output is correct and follows the requested format.
- No rule violations occur, or if they do, the system catches and fixes them.

If all tests pass, the bundle integration can be considered successful. Remember to also test edge cases (no relevant RAG data found, user asks multiple things at once, etc.) and monitor the model's behavior under those conditions.

The Adoption Playbook ensures anyone adopting this open bundle has a clear roadmap. The ASCII diagram concisely shows the flow of data and instructions through the system. The test cases serve as both verification and tutorial examples, covering logic-heavy tasks (interview, revision, narrative) and content-heavy tasks (factual QA, diagrams). The emphasis on expected behavior and output format in each case helps implementers know what to aim for and how to judge success.

Finally, we will give a bonus snippet of pseudocode for a CLI validation tool.

Bundle Validation Pseudocode (Bonus)

To facilitate automated validation of a bundle, here's a CLI-style pseudocode example. This script would check that the manifests conform to schemas and that all files are present and correct:

```
#!/usr/bin/env python3

import os, sys, json, hashlib
from jsonschema import validate

# Load schemas (assuming schemas are in ./schemas directory)
rag_schema = json.load(open("schemas/rag_manifest.schema.json"))
ris_schema = json.load(open("schemas/ris_manifest.schema.json"))
pattern_schema = json.load(open("schemas/process_pattern.schema.json"))

# Paths (could be passed as args in real script)
rag_index_path = "rag/index.json"
ris_index_path = "ris/index.json"

# Function to compute SHA256
def sha256_file(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(8192), b''):
            h.update(chunk)
    return h.hexdigest()

# Validate RAG manifest
rag_index = json.load(open(rag_index_path))
try:
    validate(instance=rag_index, schema=rag_schema)
    print("RAG manifest schema validation: OK")
except Exception as e:
    print(f"RAG manifest schema validation: FAIL ({e})")
    sys.exit(1)

# Check each document file exists and matches checksum
for doc in rag_index.get("documents", []):
```

```

file_path = os.path.join("rag", doc["file"])
if not os.path.isfile(file_path):
    print(f"ERROR: Document file missing: {file_path}")
    sys.exit(1)
if "checksum" in doc:
    actual_hash = "sha256-" + sha256_file(file_path)[:8] # using first 8
chars for brevity
    expected_hash = doc["checksum"][:15] # comparing prefix (assuming
checksum stored with sha256- prefix)
    if not actual_hash.startswith(expected_hash):
        print(f"WARNING: Checksum mismatch for {file_path}! Expected
{doc['checksum']}, got {actual_hash}")
    else:
        print(f"Checksum OK for {file_path}")

# Validate RIS manifest
ris_index = json.load(open(ris_index_path))
try:
    validate(instance=ris_index, schema=ris_schema)
    print("RIS manifest schema validation: OK")
except Exception as e:
    print(f"RIS manifest schema validation: FAIL ({e})")
    sys.exit(1)

# Check each referenced file in RIS exists
categories = ["patterns", "prompts", "templates", "checklists", "behavior"]
for category in categories:
    for entry in ris_index.get(category, []):
        file_path = os.path.join("ris", entry["file"])
        if not os.path.isfile(file_path):
            print(f"ERROR: {category.rstrip('s').capitalize()} file missing:
{file_path}")
            sys.exit(1)
        else:
            print(f"Found {file_path}")

# (Optional) Validate pattern files against pattern_schema
for entry in ris_index.get("patterns", []):
    path = os.path.join("ris", entry["file"])
    pattern = json.load(open(path))
    try:
        validate(instance=pattern, schema=pattern_schema)
        print(f"Pattern {entry.get('id')} schema validation: OK")
    except Exception as e:
        print(f"Pattern {entry.get('id')} validation FAIL: {e}")

print("Bundle validation completed.")

```

In a real scenario, this script could be invoked as:

```
$ python validate_bundle.py
```

It would output the status of each check. For example:

```
RAG manifest schema validation: OK
Checksum OK for rag/data/company_policy_handbook.md
Checksum OK for rag/data/AI_research_trends.md
...
RIS manifest schema validation: OK
Found ris/patterns/example_process.json
Found ris/prompts/interview_flow.md
...
Pattern interview_flow schema validation: OK
Pattern text_revision schema validation: OK
Bundle validation completed.
```

Any errors or warnings would be printed accordingly. This helps bundle maintainers and adopters quickly spot issues (like missing files or mismatched versions). Note that in this pseudocode, we shortened the hash check for brevity; in practice you'd compare full hashes. Also, we assume the presence of `jsonschema` library for schema validation – an open source library.

This kind of tool ensures that the bundle remains **self-consistent**, which is especially useful as the project grows. It's entirely vendor-neutral and can be extended (e.g., to check YAML files formatting, or to run some basic QA on example outputs).

Conclusion: By following this comprehensive bundle structure for RAG and RIS, developers can integrate external knowledge and complex instruction flows into LLM applications in a maintainable, secure, and open way. The provided repository layouts, manifests, schemas, and examples serve as a foundation that can be customized for various domains and use cases, all while adhering to principles of instruction-data separation for safety ² and robust software practices for reliability.

¹ Exploring Retrieval-Augmented Generation (RAG) Architectures: Enhancing AI with External Knowledge | by Gokcer Belgusen | Medium

<https://medium.com/@gokcerbelgusen/exploring-retrieval-augmented-generation-rag-architectures-enhancing-ai-with-external-knowledge-431063cb1810>

² ⁵ Can LLMs Separate Instructions From Data? And What Do We Even Mean By That?

<https://arxiv.org/html/2403.06833v3>

³ ⁶ ⁷ Semantic Versioning 2.0.0 | Semantic Versioning

<https://semver.org/>

4 ○ Guardrails in Generative AI: Keeping Your LLMs Safe and Reliable | by Sangeethasaravanan | Jun, 2025
| Medium

<https://sangeethasaravanan.medium.com/%EF%B8%8F-guardrails-in-generative-ai-keeping-your-llms-safe-and-reliable-b0679c3849ff>